

Erfahrungsbericht

FlowHub mit KI bauen — Learnings & Harness

Andreas Imboden · CAS AI-Assisted Software Engineering · FFHS FS26

Wie war es wirklich, ein Full-Stack-Projekt fast vollständig mit KI zu bauen?



Worum es geht

Nicht das Produkt — die **Arbeitsweise**.

- **~85-95 %** des Codes KI-generiert — aber das ist nicht der eigentliche Punkt
- **Kernthese:** Gute KI-Entwicklung ist kein „Prompt rein, Code raus“, sondern ein **Harness** aus Instruktionen, Skills, Pipeline und Disziplin
- Die Rolle des Menschen verschiebt sich vom Tippen zum **Architekten & Reviewer**

Der Harness – Überblick

Die Werkzeugkette, die die KI gesteuert hat:

Ebene	Werkzeug
Agent	Claude Code (interaktiv) · Codex / Copilot ergänzend
Konventionen	<code>ai-instructions</code> (base + <code>dotnet-blazor</code>) → <code>CLAUDE.md</code>
Workflows	eigene Skills: <code>/ui-*</code> , <code>/flowhub-*</code> , <code>/commit</code> , <code>/push</code>
Methode	Superpowers: Brainstorm → Spec → Plan → Subagent → Review
Automatisierung	<code>claude-pipeline</code> (Issue→PR) · <code>examiner-sim</code> (Grading)
Disziplin	Context-Hygiene: Logs-via-File · <code>/clear</code> -Schritte

Geschichtetes Agent-Onboarding

KI-Agents brauchen ein gepflegtes Onboarding-Dokument — wie ein neuer Teamkollege.

- `CLAUDE.md` — harte Projektregeln
- `.ai/base-instructions.md` — kanonische Konventionen (stack-agnostisch)
- `.ai/cas-instructions.md` — CAS-Kontext (Blöcke, Bewertungskriterien)
- + **Stack-Overlay** `dotnet-blazor` — z. B. **SemVer** · **Conventional Commits** · **12-Factor** · **TDD**

Eigene Skills > Prompt-Templates: der Agent erkennt selbst, wann anzuwenden, und folgt einer geprüften Checkliste statt zu improvisieren — befolgt von Claude Code, Codex, Copilot **und** Gemini gleichermassen.

Nicht „Prompt rein, Code raus“ — eine Pipeline

Jeder grössere Baustein lief durch denselben strukturierten Ablauf:

Brainstorm → Spec → Plan → Subagent-Implementierung → Review ×2

1. **Brainstorming** — Design als A/B/C-Entscheidungen, jede mit Begründung festgehalten
2. **Spec + Plan** — schriftliches Design, dann TDD-geordneter Aufgabenplan (**28 Specs/Plans** in `docs/superpowers/`)
3. **Subagenten** — pro Task ein frischer Implementierer (Test-First)
4. **Review ×2** — Spec-Konformität, dann Code-Qualität — bevor etwas in `main` geht

Context-Hygiene — das unterschätzte Thema

Der Kontext ist das knappste Gut. Zwei Disziplinen brachten am meisten:

1 · Logs via File — grösster Token-Fresser waren Console-, Test- und Build-Streams. Statt alles ins Conversation-Window: in eine **Datei** schreiben, gezielt mit `Read offset/limit` oder `grep` holen. → **~5-10x weniger Tokens** pro Debug-Session.

2 · /clear-Schnitte — Spec → `/clear` → Plan → `/clear` → Implement.
Jede Phase hinterlässt ein **Artefakt auf Disk**; der Dialog-Ballast wird verworfen.

Was zwischen Phasen weiterleben muss, gehört in eine Datei — nicht in den Chat.

Werkzeug-Disziplin

Skills kosten System-Prompt-Tokens — bevor die erste Nachricht gelesen wird.
Zu viele geladene Skills → unschärfere Trigger, knapperes Budget.
→ in **thematische Plugins** splitten, nur das Relevante aktivieren
(`flowhub` , `cas-aise-*` , schmales `superpowers` -Set).

Code-Exploration — das richtige Werkzeug je Frage:

`rg` (lexikalisch, schnell) → **LSP** (Symbole, Renames, Inheritance) → **grepai**
(semantisch)

Heuristik: erst `rg` ; liefern **3 Anläufe** nichts, ist die Frage nicht lexikalisch, sondern semantisch.

Automatisierung — KI prüft KI

claude-pipeline (GitHub Actions) — autonome Issue-Implementierung: Issue mit **ai-implement** labeln → Branch + Draft-PR, mit Retry-Policy (Rate-Limit / Transient / **max-turns**).

examiner-sim (Multi-Agent-Workflow) — baut die Abgabe-PDFs, benotet sie gegen die **Moodle-Rubrik** mit einem Agenten-Panel und übt die Live-Demo.

*Der Mensch schreibt nicht mehr jede Zeile — er definiert die Leitplanken und lässt **KI-gestützte Prüfungen** finden, was die KI übersieht.*

KI-Anteil in Zahlen (Block 4: Persistenz)

Artefakt	Zeilen	KI-generiert	Mensch	KI %
Entity-Klassen + Configs (14)	~250	~225	~25	90 %
Repository-Implementierungen (6)	~350	~315	~35	90 %
Integrationstests (16)	~230	~210	~20	91 %
Docker Compose	~50	~40	~10	80 %
... Services · Filter · Refactor	~130	~112	~18	86 %
Gesamt	~1010	~902	~108	~89 %

Über alle Blöcke: **~85-95 % des Codes KI-generiert** — der Mensch-Anteil ist klein, aber **hochwertig**.

Wo die KI glänzt — und wo nicht

Glänzt — repetitiv & gut spezifiziert

7 strukturgleiche `IEntityTypeConfiguration<T>`, EF-Migrations, Refit-Interfaces, CI-YAML;
16 Integrationstests gegen echtes PostgreSQL (Testcontainers) — **alle grün beim ersten Lauf.**

Scheitert — wo Domäne & Performance zählen

- **N+1-Blindheit** — `ListAsync` ohne `.Include(c => c.Tags)`
- **CASCADE überall** — owned vs. referenced ist eine menschliche Domänen-Entscheidung
- **Veraltete Versionen** — Trainingsdaten hinken neuen Releases hinterher
- **Feature-Drift** — Scope-Disziplin muss vom Menschen kommen

Der Smoke-Test-Moment

Die KI schrieb den ganzen Deployment-Stack. Dann lief **ein** Befehl:

`make smoke-prod` — End-to-End-Probe des laufenden Stacks.

An einem Nachmittag fand er 5 reale, latente Bugs:

- `.editorconfig` fehlt im Build-Image → Build bricht mit `TreatWarningsAsErrors` ab
- Compose-Env-Casing `${EMBEDDINGS__APIKEY}` $\neq$ `Embeddings__ApiKey` → Embeddings still no-op
- Leerstring-Modellname → `AssertNotNullOrEmpty`-Crash beim Start
- Mistral lehnt das `dimensions`-Feld ab → 422
- Passbolt-Refs vom Makefile überschattet → KI-Call erreichte nie den Provider

*KI schrieb den Code. Eine **KI-gestützte Prüfung** fand, was die KI übersah.*

Fazit — was bleibt

Nicht einzelne Tools, sondern **Disziplinen**:

- gepflegte, **geschichtete Agent-Instructions** (wiederverwendbar über Projekte)
- **eigene Skills** für wiederkehrende Workflows
- **Context-Hygiene** über Datei-Artefakte + `/clear`-Schnitte
- der **Spec → Plan → Implement**-Rhythmus mit Review-Gates

Der Mensch bleibt Architekt und Reviewer — die KI wird zum Verstärker.

Details: docs/ai-usage.md · vault/Projektarbeit/Learnings.md · Danke — Fragen?